QA Testing Guide

AI Career Growth & Talent Intelligence Platform — Backend (Phase 1 & Phase 2)
For anyone picking up QA/testing duties on this API

---

1. What you're testing

A FastAPI backend covering Phase 1 (Authentication, Candidate Portal, Resume Upload, Resume Parser) and Phase 2 (Candidate Profile Engine, Benchmark Repository, Matching Engine) of the platform. Six modules are implemented and testable end-to-end today:

- **Module 1** — Authentication & Access Management (JWT access/refresh, RBAC, password reset, admin user management)
- **Module 2** — Candidate Management System (profile, skills, education, experience, resume upload/versioning)
- **Module 3** — Resume Intelligence Engine (text extraction + rules-based structured parsing)
- **Module 4** — Candidate Profile Database (paginated, filterable search for company staff/admins)
- **Module 5** — Benchmark Repository (managed by company staff and admins)
- **Module 6** — Benchmark Matching Engine (Match Score, Readiness Score, Gap Summary)

Since the last QA pass: 5 issues from the prior QA report were fixed (connection-pool exhaustion under load, a 404 contract inconsistency, international phone extraction, skills-dictionary recall, and a missing provisioning script — Section 10), and two functional changes landed: the role model is back to 5 distinct roles with an admin role-reassignment endpoint (Section 4), and candidate search is now paginated and returns each candidate's name/email (Section 5.3).

Phases 3-6 (Gap Analysis / Recommendations / Roadmap, Recruiter & HR portals, Employer portal, Consultation & Learning marketplace) are **not built yet** — do not file bugs against endpoints that don't exist yet; check the endpoint list in Section 5 first.

2. Tech stack (so error messages make sense)

- **FastAPI** + SQLAlchemy 2.0 + Alembic — the API framework and DB layer
- **PostgreSQL + pgvector** — relational data, plus vector similarity for matching
- **JWT** (access + refresh tokens) — stateless auth, no server-side sessions
- **Local, free parsing/embeddings** — OpenAI is intentionally disabled right now (no API key configured). Resume parsing uses a regex/keyword rules engine only; embeddings use scikit-learn's HashingVectorizer, not an LLM. This is expected, not a bug — see Section 7.
- **Resume storage** — files are stored as blobs directly in Postgres, not S3 (also intentional for now).

3. Getting the environment running

3.1 Start Postgres (Docker Desktop must be running first):

```
docker compose up -d db
```

Maps to host port **5433**, not 5432 — 5432 is commonly already in use locally.

3.2 Python environment:

```
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
```

3.3 Copy env file and run migrations:

```
cp .env.example .env
.venv\Scripts\python -m alembic upgrade head
```

3.4 Run the server:

```
.venv\Scripts\python -m uvicorn app.main:app --reload
```

Interactive docs at **http://127.0.0.1:8000/docs** (Swagger UI).

Port-in-use error? "WinError 10013" or "address already in use" on port 8000 almost always means another uvicorn instance (yours or someone else's test run) is still bound to that port. Check for a stray python process before assuming the app itself is broken.

4. Roles and how to get a test account for each

There are **5 roles**: **candidate**, **recruiter**, **hr_reviewer**, **employer**, and **administrator**. (An intermediate build of this backend briefly collapsed recruiter/hr_reviewer/employer into a single "employee" role — that was reverted; if you have test data from that window, those accounts were remapped to `recruiter` as a default, and can be individually reassigned — see "Reassigning a user's role" below.)

Role summary

- **candidate** — self-registers via `POST /auth/register`. Owns and edits their own profile, skills, education, experience, and resumes. Can read (not manage) benchmarks and compute/view only their own matches.
- **recruiter / hr_reviewer / employer** — company staff, referred to collectively as "company staff" below since they currently share identical permissions. **Cannot self-register** — created only by an administrator (Section 5.2). Each can view **every** candidate's profile, search/filter candidates, compute/view matches for any candidate, and create/edit/delete benchmarks. None can edit a candidate's own profile data (skills/education/experience stay candidate-owned). The three labels exist for organizational/reporting clarity — permissions do not currently differ between them.
- **administrator** — everything company staff can do, plus user management (create/list/activate/deactivate any user, and reassign any user's role).

Getting a candidate account

Just register normally: `POST /auth/register` with `"role": "candidate"` (or omit role — it's the default). This is the only role the public endpoint will accept for a brand-new email.

Getting an administrator account

Admins are designated by an email allowlist, not a role field. The server has an `ADMIN_EMAILS` env var (comma-separated). If you register with an email on that list, you are **automatically granted the administrator role** regardless of what role you passed — every other email is rejected if it tries to register as anything other than candidate. Ask whoever manages the environment which email(s) are on the allowlist, then simply `POST /auth/register` with that email like any candidate signup.

If you need a DB-level admin outside the normal register flow (e.g. bootstrapping a fresh environment with no allowlisted email yet), the provisioning script still works:

```
python provision_admin.py <email> <password> ["Full Name"]
```

Getting a recruiter / hr_reviewer / employer account

Company staff cannot self-register under any circumstances — `POST /auth/register` rejects any role except candidate with a 422, and there is no public staff-signup path. An administrator must create the account:

```
POST /api/v1/admin/users
Authorization: Bearer <admin access token>

{"email": "staff@company.com", "password": "...", "full_name": "...", "role": "recruiter"}
```

`role` defaults to `recruiter` if omitted, but accepts any of the 5 roles — so this endpoint can also create additional candidates or administrators if needed. Accounts created this way are marked verified immediately (no email confirmation step exists in this backend).

Reassigning a user's role

An administrator can change any existing user's role after the fact — useful for correcting a batch of staff created as the default `recruiter` into the right sub-role:

```
PATCH /api/v1/admin/users/{user_id}/role
Authorization: Bearer <admin access token>

{"role": "hr_reviewer"}
```

An admin cannot change their own role this way (400 if attempted — prevents accidentally locking yourself out of admin access). Reassigning a user **into** `candidate` automatically creates a candidate profile for them if they don't already have one; reassigning a candidate **out of** that role leaves their existing profile data in place rather than deleting it.

Security note: `POST /api/v1/auth/register` will **reject** `"role": "recruiter"`, `"hr_reviewer"`, `"employer"`, or `"administrator"` with a 422 (unless the email is on the admin allowlist, in which case it's silently upgraded to administrator instead of rejected). This is intentional — do not report either behavior as a bug.

5. Full endpoint reference

All routes are prefixed with `/api/v1` unless noted. "Auth required" lists which role(s) may call it; "any" means any authenticated role.

5.1 Authentication — /auth

Method	Path	Auth required	Notes
POST	`/auth/register`	none (public)	JSON body. role defaults to candidate; any other role is rejected (422) unless the email is on the admin allowlist (auto-granted administrator instead).
POST	`/auth/login`	none (public)	form-urlencoded (username, password), NOT JSON. Returns access + refresh token.
POST	`/auth/refresh`	none (needs refresh token)	JSON {refresh_token}. Old token is revoked on use — reuse gives 401.
POST	`/auth/logout`	none (needs refresh token)	JSON {refresh_token}. Revokes it.
GET	`/auth/me`	any authenticated	Returns current user.
POST	`/auth/password-reset/request`	none (public)	JSON {email}. Always 202, even for unknown emails (no enumeration).
POST	`/auth/password-reset/confirm`	none (needs reset token)	JSON {token, new_password}. Token isn't emailed yet — read from DB (see Section 8).

5.2 Administration — /admin (administrator only)

Method	Path	Auth required	Notes
POST	`/admin/users`	administrator	JSON {email, password, full_name, role}. role defaults to recruiter if omitted; accepts any of the 5 roles. This is the only way to create recruiter/hr_reviewer/employer accounts.
GET	`/admin/users`	administrator	Query params: role?, limit, offset. Lists all users, optionally filtered by role.
PATCH	`/admin/users/{user_id}/role`	administrator	JSON {role}. Reassigns an existing user's role. 400 if targeting your own account. Reassigning into candidate auto-creates a profile if missing.
PATCH	`/admin/users/{user_id}/deactivate`	administrator	Sets is_active=false. Cannot deactivate your own account (400 if attempted).
PATCH	`/admin/users/{user_id}/activate`	administrator	Re-enables a deactivated user.

5.3 Candidate Management — /candidates

Method	Path	Auth required	Notes
GET	`/candidates/me`	candidate	Own profile incl. skills/education/experience.
PUT	`/candidates/me`	candidate	Update profile fields — all optional, partial updates supported.
POST	`/candidates/me/skills`	candidate	JSON {name, proficiency?, category?}.
DELETE	`/candidates/me/skills/{skill_id}`	candidate	

Method	Path	Auth required	Notes
POST	`/candidates/me/education`	candidate	JSON {degree?, field_of_study?, institution?, start_date?, end_date?, grade?}.
DELETE	`/candidates/me/education/{education_id}`	candidate	
POST	`/candidates/me/experience`	candidate	JSON {title?, company?, industry?, start_date?, end_date?, is_current?, description?}.
DELETE	`/candidates/me/experience/{experience_id}`	candidate	
GET	`/candidates`	recruiter, hr_reviewer, employer, administrator	Query params: industry, functional_area, experience_level, min/max_experience_years, skill, limit, offset. Candidate role gets 403. Returns a paginated envelope, not a bare list — see note below.
GET	`/candidates/{candidate_id}`	recruiter, hr_reviewer, employer, administrator	404 if candidate_id doesn't exist.

Search response shape changed (see Section 10): `GET /candidates` now returns `{items, total, limit, offset}` instead of a bare JSON array — `items` is the page of results, `total` is the full matching count regardless of pagination. Each item (and the single-candidate `GET /candidates/{id}` response) now also includes `full_name` and `email` pulled from the account, not just profile fields — useful for identifying who you're looking at in a candidate list.

5.4 Resume Intelligence — /resumes

Method	Path	Auth required	Notes
POST	`/resumes/upload`	candidate	multipart/form-data, field name 'file'. pdf/doc/docx only (400 otherwise), 10MB max (400 otherwise). Parsing runs in background — status starts 'pending'.
GET	`/resumes`	candidate	Lists all versions for the current candidate.
GET	`/resumes/{resume_id}`	candidate	Returns parsing_status, parsing_method, parsed_data.
GET	`/resumes/{resume_id}/download`	candidate	Streams the raw file back (not a presigned URL — storage is Postgres, not S3).
POST	`/resumes/{resume_id}/reparse`	candidate	Re-runs parsing on an existing upload.
DELETE	`/resumes/{resume_id}`	candidate	

After a successful upload, poll `GET /resumes/{id}` until `parsing_status` is `completed` (usually 1-3 seconds) before asserting on parsed_data or on the candidate profile having been auto-populated.

5.5 Benchmark Repository — /benchmarks

Method	Path	Auth required	Notes
POST	`/benchmarks`	recruiter, hr_reviewer, employer, administrator	JSON — name, category, level required; required_skills/preferred_skills/certifications etc. optional lists. (Company staff manage benchmarks, not just admins — see Section 4.)
GET	`/benchmarks`	any authenticated	Query params: category, level, functional_area, is_active. Candidates can read but not manage benchmarks.
GET	`/benchmarks/{benchmark_id}`	any authenticated	

Method	Path	Auth required	Notes
PUT	`/benchmarks/{benchmark_id}`	recruiter, hr_reviewer, employer, administrator	All fields optional (partial update).
DELETE	`/benchmarks/{benchmark_id}`	recruiter, hr_reviewer, employer, administrator	Soft-delete — sets is_active=false, does not remove the row.

5.6 Matching Engine — /matching

Method	Path	Auth required	Notes
POST	`/matching/me/compute`	candidate	Optional ?benchmark_id=... to target one; otherwise computes against all active benchmarks (filtered by the candidate's industry if set).
GET	`/matching/me`	candidate	Lists previously computed matches.
POST	`/matching/candidates/{candidate_id}/compute`	recruiter, hr_reviewer, employer, administrator	
GET	`/matching/candidates/{candidate_id}`	recruiter, hr_reviewer, employer, administrator	404 if candidate_id doesn't exist (fixed — see Section 10, BK-2).

Each match result includes `match_score`, `readiness_score`, `semantic_similarity`, matched/missing skill and certification lists, `experience_gap_years`, and a structured `gap_summary` object.

6. Testing via Swagger UI (/docs) — things that trip people up

- **Login is a form, not JSON.** POST /auth/login renders as a username/password form in Swagger — don't paste a JSON body into it.
- **Authorize button expects the raw token, no "Bearer" prefix.** Click the padlock icon top-right, paste only the JWT string. Swagger adds "Bearer " automatically. Pasting "Bearer eyJ..." yourself produces "Bearer Bearer eyJ..." and every request 401s.
- **Switching users requires re-authorizing.** Click Authorize -> Logout -> log in as the other user -> Authorize again with the new token. There's no per-request override in the Swagger UI.
- **Resume upload is a file picker**, not a JSON/text box — click "Choose File" under Try it out.
- **A 401 right after a successful login** almost always means a copy-paste error in the token (extra "Bearer", truncated string, or using the refresh_token where an access_token is expected).

7. Known and expected behavior (not bugs)

- **parsing_method is always "rules", never "llm" or "hybrid".** The OpenAI-based LLM fallback for low-confidence resume parses is intentionally disabled (no OpenAI account). Rules-only parsing runs for every resume, including messy ones it can't fully structure.
- **Legacy .doc files parse poorly.** Only .pdf and .docx get proper text extraction; .doc (old binary Word format) gets a best-effort byte-level strip, no real text layer. Prefer .pdf/.docx test fixtures.
- **semantic_similarity uses local word/bigram hashing, not an LLM embedding.** It correctly rewards literal keyword overlap (e.g. "Python" appearing in both candidate and benchmark text) but will not catch synonyms (e.g. "ML" vs "machine learning"). Do not expect GPT-level semantic understanding here.
- **Skills/phone extraction is still keyword/pattern-based, not NLP.** The skills dictionary and phone regex were both broadened in the latest fix pass (Section 10) — international numbers now extract and many more languages/stacks are recognized — but it is still a fixed dictionary/pattern, not free-text understanding. A skill or number format outside what's listed will still be missed; that's a coverage gap to note, not grounds to fail a ticket on its own.
- **No rate limiting is implemented.** Rapid repeated requests will not get 429s. This is a known gap for pre-production, not something to fail a ticket over unless you were explicitly asked to test it.
- **Resume files live in Postgres, not S3.** Download uses a direct byte-streaming endpoint, not a presigned URL — don't expect an S3-style URL in the response.
- **is_verified exists on the user model but nothing currently enforces it at login.** An unverified user can still log in. Flag this if you think it should block login, but it may be an intentional product decision.
- **The first matching/compute call for a candidate is slower than subsequent ones.** Building the candidate's embedding happens lazily on first use; expect a one-time latency cliff on cold start, then much faster warm calls. Not a failure.

8. Known gaps — do not re-report these, they're already tracked

- Password reset token is never emailed — it must be read directly from the database to complete the flow end-to-end.
- No MIME-sniffing or malware scanning on resume upload — only the file extension is checked.
- No OCR — scanned/image-only PDFs (no real text layer) will fail to parse meaningfully.

- Single-instance/single-worker only — the connection pool was sized for meaningfully higher concurrency (Section 10, BK-1), but multi-process/horizontal scaling has not been tested.
- Resume upload holds a DB connection while writing the file blob and updating prior-version status, adding latency under load (this contributed to BK-1 and has been mitigated by the pool resize, but the handler itself hasn't been restructured to release the connection earlier).
- Phases 3-6 of the platform (gap analysis, recommendations, employer/consultation portals, learning marketplace) do not exist as backend endpoints yet — Phase 3's modules exist as a standalone, not-yet-integrated package; ask if you need details on testing that separately.

9. Suggested end-to-end test flow

1. Register a candidate via /auth/register. Confirm registering with role=recruiter/hr_reviewer/employer/administrator is rejected (422), UNLESS you're using an allowlisted admin email (in which case it should succeed and come back as role=administrator).
2. Log in as the candidate. Confirm /auth/me returns the right user, and an unauthenticated request 401s.
3. Log in as an admin (allowlisted email, or provisioned via provision_admin.py). Use POST /admin/users to create a recruiter account (role defaults to recruiter if omitted). Confirm a candidate or the new recruiter both get 403 if they try to hit /admin/users themselves.
4. As the admin, PATCH /admin/users/{recruiter_id}/role to hr_reviewer, then confirm the account still logs in and retains company-staff permissions under the new label. Confirm the admin gets 400 trying to change their own role.
5. Build out the candidate profile: PUT /candidates/me, add 3-5 skills, one education entry, one experience entry.
6. Upload a real .pdf or .docx resume. Poll GET /resumes/{id} until parsing_status is completed.
7. Re-check GET /candidates/me — confirm skills/certifications/education/experience from the resume were merged in, and profile_completeness increased.
8. Confirm the recruiter/hr_reviewer/employer account CAN search/view that candidate (GET /candidates, GET /candidates/{id}) and the candidate role CANNOT (403). Confirm GET /candidates returns the paginated {items, total, limit, offset} shape and each item includes full_name/email.
9. As company staff (or admin), create 1-2 benchmarks relevant to the candidate's skills via POST /benchmarks. Confirm the candidate role gets 403 on the same call but CAN still GET /benchmarks to read them.
10. As the candidate, POST /matching/me/compute and inspect match_score, readiness_score, and gap_summary for correctness given the skills/experience you set up.
11. As company staff, compute/view matches for that same candidate via the /matching/candidates/{id} endpoints and confirm the results match what the candidate saw. Also confirm GET /matching/candidates/{random-unknown-id} now returns 404, not 200 [].
12. Try the negative/edge cases: wrong password, expired/reused refresh token, oversized file upload, unsupported file type, unknown candidate/benchmark IDs, cross-role access attempts.
13. If load-testing, fire 50-100 concurrent requests at a DB-backed endpoint (e.g. POST /matching/me/compute) and confirm you get 200s (or at worst a graceful 503 with Retry-After), not raw 500s.

10. Changelog since the last QA pass

The previous QA report (Backend Integration QA Report) found 3 filed bugs and 3 observations across Modules 1-6, plus a separate ask to redesign the role model. All of it has been addressed:

Bug fixes

- **BK-1 (Major) — DB connection-pool exhaustion, HTTP 500 under load.** Root cause was SQLAlchemy's default pool (5 + 10 overflow = 15 connections) being far smaller than Starlette's ~40-thread pool, so concurrent requests queued until pool_timeout and then raised an unhandled TimeoutError as a raw 500. Fixed: pool resized to 20 + 40 overflow (configurable via DB_POOL_SIZE / DB_MAX_OVERFLOW / DB_POOL_TIMEOUT in .env), and a global exception handler now maps SQLAlchemy TimeoutError/OperationalError to a graceful 503 with a Retry-After header instead of a raw 500. Re-tested at 60 concurrent DB-heavy requests: 0 errors (previously 72% failed at 50 concurrent).
- **BK-2 (Minor) — GET /matching/candidates/{candidate_id} returned 200 [] for a non-existent candidate.** Fixed: the endpoint now checks the candidate exists first and returns 404 if not, matching the behavior of the compute endpoint and GET /candidates/{id}.
- **BK-3 (Minor) — resume phone extraction missed international formats.** The regex was rewritten to match either a leading-plus international number or a number with 3+ digit groups, then validates by digit count (8-15, per E.164) to avoid false positives like date ranges. Re-tested against +44, +46, +971, +82, +1, and common US formats — all now extract correctly; "2016 - 2020"-style ranges are still correctly rejected.

Observations addressed

- **O-1 — skills-dictionary recall ~48%.** The dictionary was expanded from ~30 to ~115 entries, adding mobile (iOS, Android, Swift, Flutter), additional languages (Rust, Scala, Kotlin, Go, C++, C#, PHP, R), big-data (Spark, Kafka, Hadoop, Hive, Airflow), and more cloud/DevOps/ML terms. The word-boundary matching was also fixed so tokens like "c++", "c#", and ".net" match correctly without false-matching substrings (e.g. "java" no longer matches inside "javascript"). This is still a fixed dictionary, not free-text NLP — see Section 7.
- **O-3 — provision_admin.py and .env.example referenced but missing from the repo.** provision_admin.py has been added at the repo root (now also supports promoting an existing user to administrator, and creating additional admins). .env.example was already present in the repo — if you couldn't find it, confirm you're looking at the right checkout.
- **O-2 (upload latency ~3.4s)** — not yet restructured; see Section 8, still a known gap re: the DB connection being held during the blob write.

Role model: 5 roles restored, with admin role management

- An earlier revision of this backend collapsed recruiter/hr_reviewer/employer into a single "employee" role. That has been **reverted** — the product wants the 3 distinct company-staff labels back. A new migration (0003_uncollapse_roles) remaps any existing employee rows to recruiter as a default; the original sub-role was already unrecoverable from the first collapse, so this is a deliberate, documented one-way choice, not data loss beyond what already happened.
- Administrators are still granted via an ADMIN_EMAILS allowlist checked at registration time (unchanged from the prior redesign), and the DB provisioning script still works as a bootstrap fallback.
- The admin-only /admin/users endpoint (added in the prior redesign so company staff can be created without database access) still works the same way, now defaulting new accounts to role=recruiter.

- **New:** PATCH /admin/users/{user_id}/role lets an admin reassign any existing user's role — the practical tool for correcting the recruiter default after the migration, or for role changes in general (e.g. promoting a candidate to staff, or moving staff between recruiter/hr_reviewer/employer). Blocks self-role-change (400) and auto-creates a candidate profile if reassigning someone into that role.
- Permissions did not change: recruiter/hr_reviewer/employer still share identical access (view every candidate profile, search/filter candidates, compute/view any candidate's matches, create/edit/delete benchmarks — but not edit a candidate's own profile data). The 3 labels are for organizational clarity, not currently for differentiated access.

Candidate search: pagination + name/email enrichment

- GET /candidates now returns a paginated envelope {items, total, limit, offset} instead of a bare array, so company staff can build a proper paged directory UI instead of guessing whether a short result list means "no more matches" or "there's a limit you don't know about".
- Each result — in both GET /candidates and GET /candidates/{id} — now includes full_name and email pulled from the candidate's account, not just profile fields. Previously a search result had no way to identify whose profile it was without a separate lookup; this closes that usability gap.
- The total count is computed against the un-joined query specifically to avoid the skills/education/experience joinedload eager-loads inflating the count via row fan-out — worth knowing if you ever extend this endpoint, since it's an easy correctness trap to reintroduce.

Generated for internal QA handoff. If anything in this document is out of date with the running API, the code and OpenAPI schema at /docs are the source of truth — flag the discrepancy so this guide can be updated.